# Live Reolink doorbell video and audio on an ESP32-P4 display

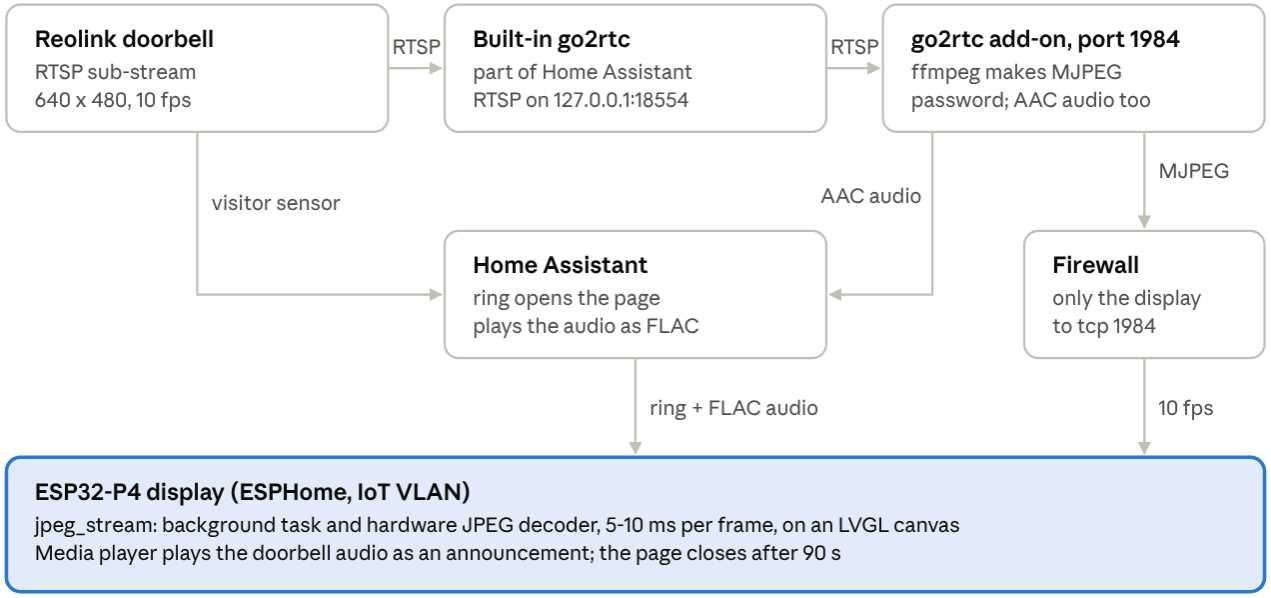
Oct 11, 2026 · @Martijn

## What you build

When someone rings the doorbell, an ESP32-P4 display opens a page with the live picture of the Reolink doorbell: 10 frames per second, under 1 s delay. At the same time the doorbell audio plays on the display's speaker. A video button shows the picture without a ring as well. From the IoT VLAN the display only needs port 1984 of go2rtc and the Home Assistant URL it already uses; the camera stays closed to it.

**The display reads one MJPEG stream; Home Assistant brings the audio**

Video, audio and the ring signal from the doorbell to the ESP32-P4 display




setup · video, audio and ring signal

The display opens one MJPEG connection to go2rtc; Home Assistant sends the ring and plays the audio.

Written for: a Reolink D340P (any Reolink with an RTSP sub-stream in Home Assistant works), a Waveshare display with an ESP32-P4 and PSRAM (1024×600, LVGL), Home Assistant OS 2026.10 with the go2rtc add-on 1.9.14, ESPHome 2026.11.0-dev and a MikroTik router as the firewall example. All code is in the repository martijnbuts/esphome-p4-doorbell.

# What you need

You need a Reolink camera in Home Assistant, the go2rtc add-on and an ESP32-P4 display that runs ESPHome with LVGL.

| Part | Requirement | Why |
|---|---|---|
| Reolink doorbell or camera | Added with the Reolink integration; a sub-stream of 640×480 | Home Assistant's built-in go2rtc serves its RTSP stream on 127.0.0.1:18554 |
| Home Assistant | HA OS or Supervised (add-ons); tested on 2026.10 | Runs the go2rtc add-on and converts the audio for the display |
| go2rtc add-on | From the AlexxIT add-on repository; tested on 1.9.14 | Turns the RTSP stream into MJPEG and AAC, behind a password |
| ESP32-P4 board with display | PSRAM (the frame buffers take about 1.5 MB), Wi-Fi, LVGL | The P4 has a hardware JPEG decoder: 5–10 ms per frame |
| ESPHome | 2026.11.0-dev with ESP-IDF 5.5 (tested) | The component uses the LVGL 9 canvas and the ESP-IDF JPEG driver |
| Media player on the display | An ESPHome `speaker` media player with an announcement pipeline | The doorbell audio plays as an announcement and pauses music |
| Router | Firewall rules between VLANs (the example is MikroTik RouterOS 7) | The display sits in an IoT VLAN and may only reach go2rtc |

Replace the values between `< >` with your own: `<ha-ip>`, `<display-ip>`, `<go2rtc-password>`, `<stream-name>`, `<camera-entity>`, `<doorbell-sensor>` and `<display-media-player>`. Make a Home Assistant backup and an export of your router configuration before you start.

# Step 1: Set up go2rtc

This step makes an MJPEG stream of the doorbell that the display can read. The go2rtc add-on takes the RTSP stream from Home Assistant's built-in go2rtc and lets ffmpeg turn it into MJPEG. Only the API port gets a password; Home Assistant itself (127.0.0.1) needs none.

1. Turn on *Preload stream* for the camera: *Settings* → *Devices & services* → *Entities*, open `<camera-entity>`, click the gear icon, switch on **Preload camera stream**. Home Assistant only offers the stream on 127.0.0.1:18554 while it is loaded.
2. Find the stream name. Home Assistant names it `<platform>_<unique_id>`. Run this in the *Terminal & SSH* add-on:

   ```
   jq -r '.data.entities[] | select(.entity_id=="<camera-entity>") | "\
   (.platform)_\(.unique_id)"' /config/.storage/core.entity_registry
   ```

   For a Reolink sub-stream you get something like `reolink_<serial>_0_sub`. That is your `<stream-name>`.
3. Add the add-on repository: *Settings* → *Add-ons* → *Add-on store* → ⋮ → *Repositories*, paste `https://github.com/AlexxIT/hassio-addons` and click **Add**. Install **go2rtc** and turn on **Start on boot** and **Watchdog**.
4. Create `/config/go2rtc.yaml` (the add-on reads it from there) with this content:

   ```
   api:
     listen: ":1984"
     username: p4display
     password: "<go2rtc-password>"
   rtsp:
     listen: "127.0.0.1:8554"   # only for ffmpeg inside go2rtc
   exec:
     allow_paths:
       - ffmpeg
   streams:
     doorbell: rtsp://127.0.0.1:18554/<stream-name>
     doorbell_mjpeg: ffmpeg:doorbell#video=mjpeg
   log:
     level: info
   ```

5. Restart the add-on and open its *Log* tab. You should see `[api] listen addr=:1984` and no errors.

Control: from a computer in your LAN, run

```
curl -u p4display:<go2rtc-password> -o frame.jpg "http://<ha-
ip>:1984/api/frame.jpeg?src=doorbell_mjpeg"
```

You should get a JPEG of the doorbell. The same command without `-u` must give `Unauthorized`.

The go2rtc web page in Home Assistant now asks for this user name and password too.

## Step 2: Open port 1984 for the display only

This step lets the display, and nothing else from the IoT VLAN, reach go2rtc. Give the display a static DHCP lease first, so `<display-ip>` never changes.

1. Open a terminal on the MikroTik (WinBox *New Terminal* or SSH).
2. Paste this. It adds the rule only once, so you can run it again safely:

```
/ip firewall filter
:if ([:len [find comment="P4 display to go2rtc"]] = 0) do={
  add chain=forward action=accept protocol=tcp src-address=<display-ip> \
    dst-address=<ha-ip> dst-port=1984 comment="P4 display to go2rtc"
}
```

3. Move the rule above the rule that drops traffic from the IoT VLAN: in WinBox drag it up in *IP → Firewall → Filter Rules*, or run

```
/ip firewall filter move [find comment="P4 display to go2rtc"]
destination=[find comment="<your-iot-drop-rule>"]
```

The order matters: RouterOS stops at the first rule that matches. Below a drop rule this rule never gets a packet.

The display also needs the Home Assistant URL that it already uses for text-to-speech. Home Assistant serves the doorbell audio from there (Step 5).

Control: run `/ip firewall filter print stats where comment="P4 display to go2rtc"`. After Step 4 the packet counter goes up while the doorbell page is open.

## Step 3: Add the jpeg_stream component

This step adds the component that shows the stream. It receives the frames in a background task and decodes them with the hardware JPEG decoder of the ESP32-P4. The main loop only swaps the canvas buffer, so touch and a wake word keep working.

1. Add this to the device configuration in ESPHome:

```
external_components:
  - source: github://martijnbuts/esphome-p4-doorbell@main
    components: [jpeg_stream]
```

2. Make sure the device has `psram:` and an `lvgl:` section. The component checks that the chip is an ESP32-P4.

The component reads two kinds of URL. It looks at the `Content-Type` of the answer:

| URL gives | What the component does | Example |
|---|---|---|
| `multipart/x-mixed-replace` (MJPEG) | Keeps one connection open; every part is a frame. Each part needs a `Content-Length` header. | `http://user:pass@<ha-ip>:1984/api/stream.mjpeg?src=doorbell_mjpeg` |
| `image/jpeg` (one picture) | Fetches the picture again and again over one kept-alive connection | The camera proxy of Home Assistant |

A user name and password in the URL are sent as HTTP Basic authentication. A frame that arrives while the previous one is not on the screen yet is skipped, which keeps the delay low.

| Option | Default | Meaning |
|---|---|---|
| `canvas` | required | The LVGL canvas to draw on; the component gives it its own buffers and size |
| `url` | none | Start URL; usually set with `jpeg_stream.start` |
| `max_width`, `max_height` | 640, 480 | Largest picture; sets the size of the two frame buffers in PSRAM |
| `max_jpeg_size` | 262144 | Largest JPEG in bytes |
| `timeout` | 5s | Network timeout; use 10s for go2rtc, which waits for a key frame |
| `min_interval` | 0ms | Minimum time between two frames |
| `on_frame`, `on_error` | none | Automations after each frame and after each error |

Actions: `jpeg_stream.start` (with an optional `url`), `jpeg_stream.stop` and `jpeg_stream.set_url`. Condition: `jpeg_stream.is_running`. In a lambda, `get_last_frame_time()` gives the decode time of the last frame and `get_dropped_count()` the skipped frames. The option stats_interval (default 0s, off) logs where the time goes; see Performance.

## Step 4: Add the doorbell page

This step adds the page that opens on a ring, as a package from the repository. It brings the page, the scripts, the fallback to Home Assistant snapshots and a button *Open doorbell camera* in Home Assistant.

Do you have the Waveshare ESP32-P4-WIFI6-Touch-LCD-7B? Then start from the complete example `example/waveshare-p4-7b.yaml` in the repository. It has the display, touch, Wi-Fi, the speaker and a home page with a video button. It compiles with ESPHome 2026.11.0-dev; its doorbell part is an English copy of the configuration that runs on the tested display. Otherwise add the package to your own configuration:

1. Put the go2rtc password in `secrets.yaml` of ESPHome:

   ```
   go2rtc_password: "<go2rtc-password>"
   ```

   Use only letters and digits: the password goes into a URL.

2. Add the package to the device configuration:

   ```
   packages:
     doorbell:
       url: https://github.com/martijnbuts/esphome-p4-doorbell
       ref: main
       files:
         - path: example/doorbell.yaml
           vars:
             doorbell_sensor: <doorbell-sensor>
             camera_entity: <camera-entity>
             ha_url: <ha-url>
             go2rtc_url: http://<ha-ip>:1984
             go2rtc_user: p4display
             go2rtc_password: !secret go2rtc_password
             go2rtc_stream: doorbell_mjpeg
             go2rtc_audio: doorbell
             media_player_id: <media-player-id>
             display_media_player: <display-media-player>
             time_id: <time-id>
             home_page: <home-page-id>
             close_after: 90s
   ```

   `<doorbell-sensor>` is the binary sensor that turns on at a ring (for a Reolink: *Visitor*). `<ha-url>` is the Home Assistant URL that the display can reach, without a trailing `/`. `<media-player-id>` and `<time-id>` are ESPHome ids in your configuration; `<display-media-player>` is that media player as an entity in Home Assistant.

3. Add a video button to your own page. It opens the page without a ring:

```
- button:
    width: 56
    height: 44
    on_click:
      - script.execute: doorbell_live
```

4. In ESPHome click **Validate**, then **Install → Wirelessly**.

The page uses two LVGL frame buffers of 640×480 in PSRAM (about 1.2 MB) and a JPEG buffer of 256 KB. They are made the first time the page opens and stay for the next time.

Control: press **Open doorbell camera** in Home Assistant and open the logs of the device. Every 5 s you should see a line like `go2rtc: 50 frames in 5 s, last frame 6 ms, 0 skipped`.

## Step 5: Turn on the doorbell audio

This step lets Home Assistant play the doorbell audio on the display. When the page opens, the display asks Home Assistant to play `http://127.0.0.1:1984/api/stream.aac?src=doorbell` as an announcement. Home Assistant fetches it from go2rtc on its own host (no password needed from 127.0.0.1), converts it to FLAC and streams it to the display. Music on the display pauses meanwhile.

1. Allow the display to call Home Assistant actions: *Settings → Devices & services → ESPHome*, click **Configure** next to the display and switch on **Allow the device to perform Home Assistant actions**.
2. Make sure the media player of the display has an `announcement_pipeline` (the complete example has one):

```
media_player:
  - platform: speaker
    id: display_media_player
    name: "Media Player"
    announcement_pipeline:
      speaker: board_speaker
      format: FLAC
      sample_rate: 48000
      num_channels: 1
```

3. The package stops the audio itself when the page closes, with `media_player.stop` and `announcement: true`. The Stop button in Home Assistant does not stop an announcement.

The audio is 1–2 s behind the picture, because ffmpeg and the media player buffer it.

Control: open the page and look at the media player in Home Assistant. It should say *Playing* within a second, and you hear the street. After **Back** it goes to *Idle*.

## Performance

On the tested display a frame takes about 3.5 ms to receive and 3.5 ms to decode. The limit is the LVGL refresh of the 640×480 picture: about 41 ms per frame on the main loop, so at most about 21 fps reach the screen. The camera delivers 10 fps, so every frame is shown.

Measured on 12 October 2026 on the Waveshare ESP32-P4-WIFI6-Touch-LCD-7B (360 MHz, LVGL `rotation: 180`, `buffer_size: 100%`) with ESPHome 2026.11.0-dev. The synthetic source is an ffmpeg test picture of 640×480 from go2rtc.

| Input | Shown | Skipped | Decode | LVGL refresh (of which flush) |
|---|---|---|---|---|
| Camera, 10 fps, with `clip_corner` | 10.0 fps | 0 % | 3.5 ms | 51.5 ms (17.4 ms) |
| Camera, 10 fps | 10.0 fps | 0 % | 3.5 ms | 41.3 ms (17.4 ms) |
| Synthetic, 30 fps, with `clip_corner` | 17.2 fps | 43 % | 4.2 ms | 57 ms (18.7 ms) |
| Synthetic, 30 fps | 21.5 fps | 29 % | 4.0 ms | 45.7 ms (19.5 ms) |
| Synthetic, 60 fps, with `clip_corner` | 16.4 fps | 73 % | 6.3–8.0 ms | 60 ms (19.5 ms) |

`clip_corner` (rounded corners) on the card around the picture cost about 10 ms per frame, so the package no longer uses it. *Flush* is the 180° rotation plus the copy into the display; the rest of the refresh is LVGL rendering the picture in software.

To measure on your own display:

1. Add `stats_interval: 10s` to `jpeg_stream`. While the page is open the log shows every 10 s:

```
Stats over 10.0 s: 10.0 fps shown (100 shown, 100 received, 0 skipped, 0
errors)
  JPEG size: avg 21.7 KB, 217 KB/s
  Receive:   avg 3.4 ms, max 10.0 ms
  Decode:    avg 3.6 ms, max 6.0 ms
  Hand-off:  avg 3.1 ms, max 30.4 ms (task -> canvas in the main loop)
  Frame gap: max 155.4 ms
  LVGL refresh: 102 x, avg 51.6 ms, max 58.2 ms (layout + render + flush)
  LVGL flush:   avg 17.4 ms per refresh (flush_cb: copy to the display)
```

2. For the time per component in the main loop and the CPU load per task, add the package `example/measure.yaml` from the repository. Remove it after measuring.
3. To find the ceiling, let go2rtc send a test picture instead of the camera, and restart the add-on:

   ```
   streams:
     doorbell_mjpeg: "exec:ffmpeg -hide_banner -v error -re -f lavfi -i
   testsrc2=size=640x480:rate=30 -c:v mjpeg -q:v 8 -f mjpeg -"
   ```

4. Leave out the 10-second windows in which `runtime_stats` prints its report: that burst stalls the main loop for about 0.5 s.

The ideas that could make it faster all target the LVGL refresh: draw the video outside LVGL with the PPA, use the LVGL PPA draw unit for image blits, avoid the 180° rotation, or use double draw buffers. docs/PERFORMANCE.md has all measurements, the CPU load per task and these ideas in detail.

## Testing

Test each part on its own, from the network the display is in, before you rely on a real ring.

| Test | How | Good if |
|---|---|---|
| go2rtc gives a picture | `curl -u p4display:<go2rtc-password> -o frame.jpg "http://<ha-ip>:1984/api/frame.jpeg?src=doorbell_mjpeg"` | `frame.jpg` shows the doorbell |
| go2rtc is closed without a password | The same command without `-u` | `Unauthorized` |

| Test | How | Good if |
|---|---|---|
| Only the display reaches port 1984 | The `curl` command from another device in the IoT VLAN | Time-out |
| Picture on the display | Press **Open doorbell camera** in Home Assistant | The page opens within 2 s; the log shows about 50 frames per 5 s |
| Low delay | Compare the clock in the camera picture with a clock next to you | Less than 1 s behind |
| Audio | Open the page and watch the media player in Home Assistant | *Playing* within 1 s; you hear the street |
| Audio stops | Press **Back** | Media player *Idle*; no consumers left in go2rtc ( `/api/streams` ) |
| Fallback | Stop the go2rtc add-on and open the page | After about 2 s the status says `(via HA)` at 3–4 fps |
| Real ring | Ring the doorbell | The page opens by itself and closes after 90 s |

Start the go2rtc add-on again after the fallback test.

## Pitfalls

These problems came up while building this.

| Problem | Cause | Solution |
|---|---|---|
| 0.4 fps, touch freezes, wake word warnings | `online_image` downloads and decodes in software in the main loop (about 2.7 s per frame) | Use `jpeg_stream`: background task and hardware decoder |
| Never more than about 4 fps | Every snapshot from the Home Assistant camera proxy takes 0.25–0.3 s, and snapshots run one after the other | Read the MJPEG stream from go2rtc |

| Problem | Cause | Solution |
| --- | --- | --- |
| `streams: hass: wrong response` in the go2rtc log | The `hass://` source of go2rtc does not work with current Home Assistant | Use `rtsp://127.0.0.1:18554/<stream-name>` |
| `method DESCRIBE failed: 404` in the go2rtc log | *Preload camera stream* is off, or the stream name is wrong | Turn on preload; look up the name with the `jq` command from Step 1 |
| `HTTP status 401` in the display log | Wrong user name or password, or a password with characters such as `@` or `:` | Use the same values as in `go2rtc.yaml`, letters and digits only |
| Display cannot connect to Home Assistant on port 8123 | The IoT VLAN may not reach Home Assistant | Use the URL the display already uses (for example the HTTPS URL on 443), or open the port for the display only |
| Audio keeps playing after **Stop** in Home Assistant | The Stop button does not stop an announcement on an ESPHome media player | Close the page on the display; the package stops it with `announcement: true` |
| The first frame takes several seconds | ffmpeg waits for a key frame of the camera | Keep `timeout: 10s`; a shorter key frame interval in the camera helps |
| `OTA with 32MB flash requires 'enable_idf_experimental_features'` | ESPHome rule for boards with 32 MB flash | Add `enable_idf_experimental_features: true` under `esp32: framework: advanced:` |
| go2rtc web page asks for a login | The API password also protects the web page | Log in with the user name and password from `go2rtc.yaml` |

Don't need audio or more than 4 fps? Then the snapshot mode alone ( `jpeg_stream` with the camera proxy URL of Home Assistant) works without go2rtc and without a firewall change.